

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA1214	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	M.Koushik Reddy	Reg. No.:	192372185
Section / Batch:	CSE-AI	Date:	25-08-2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

- Design an I/O subsystem for a real-time robotic arm controller with the following constraints:
 - Position sensors generate data every 500 μ s.
 - Control commands must be processed within 5 μ s.
 - Use priority-based interrupt handling for critical sensors.
 - Non-critical diagnostic data must be transferred using DMA.
- Construct a communication architecture for an autonomous vehicle with the following constraints:
 - Multiple cameras generate a combined data rate of 2 GB/s.
 - Safety-critical sensors require deterministic response time.
 - Use PCIe for high-bandwidth devices.
 - Ensure that low-priority devices do not affect safety-critical communication.
- Design an I/O mechanism for an industrial monitoring system with the following constraints:
 - 100 sensors continuously generate measurement data.
 - The system must support real-time fault detection.
 - Use interrupt coalescing to reduce processor overhead.
 - Critical fault signals must receive immediate interrupt service.
- Construct a high-speed network interface subsystem with the following constraints:
 - Network traffic can reach 10 Gb/s.
 - The CPU must be protected from excessive packet-processing overhead.

- Use DMA-based packet transfer between the network interface and memory.
 - Generate interrupts only after processing a configurable batch of packets.
5. Design an I/O architecture for a smart healthcare monitoring system with the following constraints:
- Patient-monitoring devices generate data at different sampling rates.
 - Emergency signals must be handled with highest priority.
 - Use nested interrupt handling for urgent events.
 - Store continuous monitoring data using DMA transfers without interrupting critical processing.

Code of Conduct:-

I M.Koushik Reddy /192372185 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Koushik Reddy

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

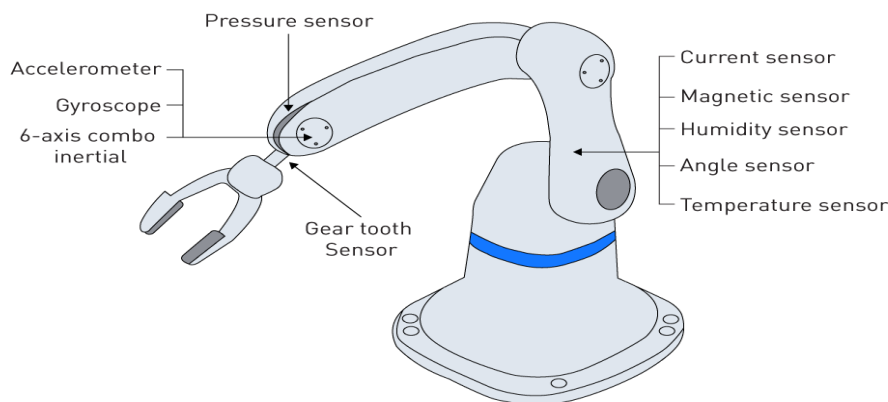
Course Coordinator Faculty Incharge:-_____

Problem 1: Real-Time Robotic Arm Controller

Given Constraints

- Position sensors generate data every 500 μs .
- Control commands must be processed within 5 μs .
- Use priority-based interrupt handling for critical sensors.
- Non-critical diagnostic data must be transferred using DMA.

1. Proposed I/O Architecture



The controller uses a vectored, priority-based interrupt architecture built around a Programmable Interrupt Controller (PIC) with at least three priority levels. The command path — encoder feedback and motor-command interrupts — is wired to the two highest priority levels so the CPU can preempt any lower-priority activity within the 5 μs budget. Position sensors are polled indirectly through a 500 μs periodic timer interrupt at the next priority tier, which is comfortably longer than the command deadline, so it never blocks a command interrupt.

Because 5 μs is too tight for a full context save/restore on many CPUs, the command ISR is kept minimal: it reads the pre-latched command register, writes the motor DAC/PWM value, and exits — heavier bookkeeping (logging, trajectory recalculation) is deferred to a lower-priority software interrupt (a 'bottom half').

Diagnostic data (temperature, current draw, vibration logs) is non-deterministic and low priority, so it is moved entirely off the interrupt path using a dedicated DMA channel that streams data from the diagnostic ADC/sensor bus straight into a memory ring buffer, raising only a single 'buffer full' interrupt at the lowest priority.

2. Block-Level Data / Control Flow

```
Position/Encoder Sensors --(500us cyclic)--> Timer/Capture Unit --IRQ(Pri-2)--> CPU
Command Source (trajectory planner) --event--> Command Latch --IRQ(Pri-1, highest)-->
CPU (5us ISR: read->compute->write PWM)
Diagnostic Sensors (temp, current, vibration) --continuous--> DMA Controller --burst--
> SDRAM Ring Buffer --IRQ(Pri-3, lowest, buffer-full only)--> CPU (bulk log/analyze)
Interrupt Controller: Pri-1 Command > Pri-2 Position-timer > Pri-3 DMA-diagnostic-
complete
```

3. Operating Sequence

1. Every 500 μs the position-timer interrupt fires; if a higher-priority command interrupt is pending or arrives mid-service, the PIC immediately preempts it.
2. A command interrupt (Pri-1) preempts everything else; the ISR executes in well under 5 μs because it performs only register read $\rightarrow$ PWM write, with no memory allocation or floating-point work.
3. Diagnostic sensors continuously push samples onto the peripheral bus; the DMA engine moves them into memory autonomously, consuming zero CPU cycles per sample.
4. Only when the diagnostic ring buffer fills does the DMA controller raise a low-priority interrupt, which the CPU services only when no Pri-1 or Pri-2 work is pending.
5. A watchdog timer cross-checks that the Pri-1 ISR always completes inside its 5 μs window; a violation trips a safety fault and freezes the arm.

4. Interrupt / DMA Priority Assignment

Interrupt Source	Priority	Max Latency	Handling
Control command	Highest (Pri-1)	$\leq 5 \mu\text{s}$	Direct ISR, register-level only
Position sensor timer	Medium (Pri-2)	$\leq 500 \mu\text{s}$	Periodic capture + short ISR
Diagnostic buffer-full	Lowest (Pri-3)	Best-effort	DMA-driven, deferred processing

5. Why This Design Meets the Constraints

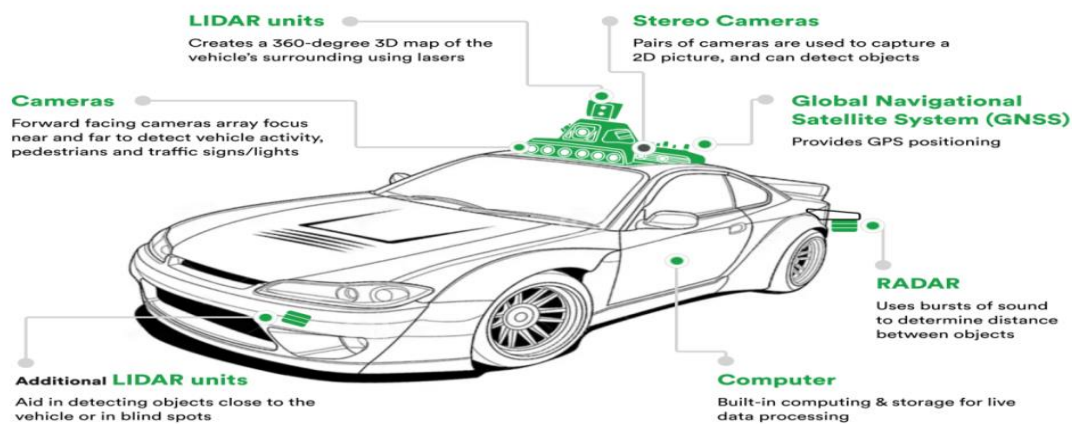
- Priority-based nested interrupts guarantee the 5 μs command deadline can always preempt the 500 μs sensor cycle.
- Keeping the command ISR to a handful of instructions makes the 5 μs bound achievable on typical embedded clock speeds (tens to hundreds of MHz).
- Offloading diagnostics to DMA removes non-critical traffic from the CPU and from the interrupt priority contention entirely.
- A single low-priority 'buffer full' interrupt (instead of one interrupt per sample) minimizes overhead from diagnostic data.

Problem 2: Autonomous Vehicle Communication Architecture

Given Constraints

- Multiple cameras generate a combined data rate of 2 GB/s.
- Safety-critical sensors require deterministic response time.
- Use PCIe for high-bandwidth devices.
- Ensure that low-priority devices do not affect safety-critical communication.

1. Proposed I/O Architecture



The architecture partitions I/O into two physically and logically separate domains: a high-bandwidth, best-effort data plane and a low-bandwidth, deterministic safety plane, so that congestion on one can never delay the other.

Cameras, LiDAR point-cloud streams, and radar raw-data feeds sit on a PCIe Gen4 x8/x16 fabric (each x4 lane group provides ~8 GB/s, comfortably covering the 2 GB/s combined camera load with headroom for LiDAR/radar). Each camera uses its own PCIe endpoint with dedicated DMA-capable buffers so frames land directly in memory without CPU copies.

Safety-critical sensors — braking, steering angle, emergency-stop, collision radar — are isolated on a separate deterministic bus (e.g., CAN-FD or a dedicated low-latency SPI/FlexRay link) with its own interrupt line at the highest CPU priority, so PCIe traffic bursts (DMA completions, link retraining) cannot delay the safety path.

PCIe Quality-of-Service features (Virtual Channels / Traffic Class tagging, plus per-device Max Payload Size and credit-based flow control) are used to prevent any single low-priority PCIe device — e.g., an infotainment or logging SSD — from starving camera DMA bandwidth.

2. Block-Level Data / Control Flow

```
Cameras (front/rear/side, ~2 GB/s total) --PCIe Gen4 x8, DMA--> Perception Memory Pool
LiDAR / Radar (high volume) --PCIe Gen4, separate lanes, DMA--> Perception Memory Pool
Braking / Steering / E-Stop sensors --dedicated CAN-FD / low-latency bus--> Safety IRQ
(highest priority) --> Safety MCU/CPU core
Infotainment / Logging SSD --PCIe (lower Traffic Class, rate-limited)--> Storage,
isolated from safety IRQ path
```

PCIe Root Complex: Traffic-Class separation + per-lane bandwidth reservation prevents low-priority devices from delaying camera/LiDAR DMA

3. Operating Sequence

1. Camera sensors continuously push frames over PCIe; each endpoint's DMA engine transfers data directly to a reserved perception memory pool with no CPU involvement.
2. The safety bus runs independently on its own physical wiring and interrupt line, so it is never queued behind PCIe transactions.
3. The PCIe root complex enforces per-Traffic-Class bandwidth reservation, guaranteeing the camera/LiDAR classes their required throughput even if a logging SSD saturates its own class.
4. A dedicated safety-domain CPU core (or lockstep core) services safety interrupts with a bounded, worst-case-analyzable latency, isolated from perception-plane load spikes.
5. The perception stack (object detection, fusion) consumes camera/LiDAR data asynchronously from memory, decoupled from the acquisition timing.

4. Interrupt / DMA Priority Assignment

Device Class	Interconnect	Priority/QoS	Determinism
Cameras / LiDAR / Radar	PCIe Gen4 (dedicated lanes)	High Traffic Class, DMA	Best-effort, high throughput
Braking / Steering / E-Stop	Dedicated CAN-FD / low-latency bus	Highest CPU IRQ priority	Deterministic, bounded worst-case
Infotainment / Logging	PCIe (shared/lower TC)	Rate-limited, lowest priority	Non-deterministic, isolated

5. Why This Design Meets the Constraints

- PCIe's per-lane bandwidth easily exceeds the 2 GB/s camera requirement, leaving margin for LiDAR/radar growth.
- Physically separating the safety bus from the high-bandwidth PCIe fabric guarantees that safety messages are never queued behind large data transfers.
- PCIe Traffic-Class/QoS tagging plus bandwidth reservation stops low-priority devices (storage, infotainment) from ever starving safety-relevant or perception-critical channels.
- DMA on every high-bandwidth endpoint keeps the CPU free to run control and safety logic instead of copying camera data.

Problem 3: Industrial Monitoring System

Given Constraints

- 100 sensors continuously generate measurement data.
- The system must support real-time fault detection.
- Use interrupt coalescing to reduce processor overhead.
- Critical fault signals must receive immediate interrupt service.

1. Proposed I/O Architecture



The 100 field sensors are grouped behind a small number of multiplexed data-acquisition modules (e.g., 4–8 sensors per module) rather than each sensor owning a dedicated CPU interrupt line, which would otherwise overwhelm the interrupt controller with 100 concurrent sources.

Routine measurement data is collected with interrupt coalescing: each acquisition module buffers several samples (or waits up to a small coalescing time-window, e.g., 1–2 ms) and raises a single interrupt for a batch, instead of one interrupt per sample. This sharply cuts the interrupt rate and the associated context-switch overhead for 100 continuously-sampling sensors.

Fault detection is split out of the coalesced path entirely: each sensor module also carries a dedicated, non-coalesced 'fault' line (out-of-range value, sensor disconnect, over-temperature) that is OR-wired into a separate, highest-priority interrupt input, so a fault is never delayed behind a batch that hasn't filled yet.

A programmable interrupt controller with at least two tiers is used: Tier-1 (fault, uncoalesced, immediate) and Tier-2 (batched measurement data, coalesced). Tier-1 always preempts Tier-2.

2. Block-Level Data / Control Flow

```
Sensor Array (100 sensors, grouped in modules of ~8) --continuous samples-->
Acquisition Module local buffer
Acquisition Module: buffer fills OR coalescing timer (~1-2ms) expires --> single
batched IRQ (Tier-2) --> CPU reads batch via block transfer
Acquisition Module: any sample out-of-range/fault --> immediate dedicated fault line -
-> Fault IRQ (Tier-1, highest, uncoalesced) --> CPU fault handler (preempts Tier-2)
```

Interrupt Controller: Tier-1 Fault >> Tier-2 Coalesced-batch (Tier-1 always preempts)

3. Operating Sequence

1. Each acquisition module continuously samples its group of sensors and accumulates readings in a local FIFO.
2. When the FIFO reaches its coalescing threshold, or a maximum wait time elapses, the module raises one Tier-2 interrupt covering the whole batch, which the CPU services with a single block read.
3. If any sample at any instant violates a threshold, the module immediately asserts the separate Tier-1 fault line, bypassing the coalescing buffer entirely.
4. The interrupt controller always services a pending Tier-1 fault interrupt before returning to or starting Tier-2 batch processing, guaranteeing immediate fault response.
5. The CPU's fault handler triggers alarms/shutdown logic and logs the offending sensor, while normal batched monitoring resumes afterward without loss of data (still buffered in the module).

4. Interrupt / DMA Priority Assignment

Signal Type	Interrupt Style	Priority	Purpose
Routine measurement batch	Coalesced (time/count threshold)	Tier-2 (normal)	Reduce CPU overhead from 100 sensors
Critical fault signal	Uncoalesced, immediate	Tier-1 (highest)	Guarantee real-time fault detection

5. Why This Design Meets the Constraints

- Grouping sensors into modules and coalescing routine interrupts keeps interrupt-controller and CPU overhead manageable at 100-sensor scale.
- A separate, uncoalesced, highest-priority fault line ensures faults are never delayed by the batching window used for normal data.
- Tier-1-over-Tier-2 preemption gives a bounded, predictable worst-case fault response time, satisfying the real-time fault-detection requirement.
- Local buffering at each module prevents data loss while the CPU is busy servicing a higher-priority fault.

Problem 4: High-Speed Network Interface Subsystem

Given Constraints

- Network traffic can reach 10 Gb/s.
- The CPU must be protected from excessive packet-processing overhead.
- Use DMA-based packet transfer between the network interface and memory.
- Generate interrupts only after processing a configurable batch of packets.

1. Proposed I/O Architecture



The Network Interface Card (NIC) uses descriptor-ring based DMA: the driver pre-allocates a ring of receive (RX) and transmit (TX) descriptors in main memory, each pointing to a packet buffer. Incoming packets at up to 10 Gb/s are written by the NIC's DMA engine directly into these buffers with zero CPU involvement.

Interrupt moderation/coalescing is configured on the NIC: instead of interrupting per packet (which at 10 Gb/s with small packets could mean millions of interrupts per second), the NIC raises one interrupt after either N packets have been DMA'd (a configurable batch size) or a small timeout elapses, whichever comes first.

The driver services interrupts using NAPI-style polling (poll-then-interrupt): on an interrupt, it disables further interrupts for that queue and pulls all available packets from the ring in a tight loop, re-enabling interrupts only once the ring is drained — bounding the interrupt rate under sustained load while still remaining responsive at low load.

Multiple RX queues with Receive-Side Scaling (RSS) spread the 10 Gb/s load across CPU cores, each with its own descriptor ring, DMA channel, and coalesced interrupt, avoiding a single-core bottleneck.

2. Block-Level Data / Control Flow

```

Network Line (up to 10Gb/s) --> NIC MAC/PHY --> NIC internal packet buffer
NIC DMA Engine --burst writes--> RX Descriptor Ring in main memory (zero-copy)
NIC coalescing logic: batch-count N reached OR timeout expires --> single MSI-X
interrupt per queue
CPU core (per RSS queue): IRQ --> NAPI poll loop drains ring --> protocol stack
processing --> re-arm interrupt
TX path: CPU stack queues packet descriptor --> NIC DMA engine reads from memory -->
transmits on wire (also batched/coalesced completion interrupts)

```

3. Operating Sequence

1. Packets arrive at the NIC and are placed by the DMA engine directly into pre-mapped memory buffers described by the RX ring, with no CPU cycles spent copying data.
2. The NIC counts DMA-completed packets; once the configurable batch size (e.g., 64 or 128 packets) is reached, or a coalescing timer expires, it raises a single interrupt for that batch.
3. The CPU's interrupt handler switches to polling mode (NAPI), draining as many ring entries as are available in one pass, which amortizes interrupt overhead across many packets.
4. Multiple queues (RSS) let several cores process the aggregate 10 Gb/s stream in parallel, each independently coalescing its own interrupts.
5. Once traffic subsides and the ring empties, the driver re-enables per-packet interrupts so latency stays low under light load — coalescing scales its effect with traffic intensity.

4. Interrupt / DMA Priority Assignment

Stage	Mechanism	Benefit
Packet ingress → memory	Zero-copy DMA into descriptor ring	Removes per-packet CPU copy overhead
Interrupt generation	Coalescing by batch-count or timeout	Reduces interrupt rate at 10 Gb/s
Interrupt servicing	NAPI poll-then-interrupt	Bounds CPU load under heavy traffic
Multi-core scaling	RSS multi-queue DMA + IRQ	Parallelizes 10 Gb/s across cores

5. Why This Design Meets the Constraints

- DMA-based zero-copy transfer is essential at 10 Gb/s, since CPU-mediated copying cannot keep up with that packet rate.
- Interrupt coalescing converts a potentially million-interrupts-per-second workload into a small, configurable number of batched interrupts, directly protecting the CPU from overload.
- NAPI-style poll/interrupt hybrid keeps latency low when traffic is light and overhead low when traffic is heavy.
- Multi-queue DMA with RSS ensures the 10 Gb/s aggregate rate is distributed rather than bottlenecked on one core.

Problem 5: Smart Healthcare Monitoring System

Given Constraints

- Patient-monitoring devices generate data at different sampling rates.
- Emergency signals must be handled with highest priority.
- Use nested interrupt handling for urgent events.
- Store continuous monitoring data using DMA transfers without interrupting critical processing.

1. Proposed I/O Architecture



Because devices sample at heterogeneous rates (ECG at ~250–1000 Hz, SpO2 at ~1 Hz, temperature at ~0.1 Hz, etc.), each device class is assigned its own interrupt priority tier proportional to its clinical urgency and data rate rather than a single shared queue, avoiding head-of-line blocking between fast and slow devices.

A nested (multi-level) interrupt controller is used: interrupt nesting allows a higher-priority interrupt (e.g., cardiac-arrest or apnea alarm) to preempt a lower-priority interrupt handler that is already executing (e.g., a routine ECG-sample ISR), rather than waiting for it to finish — critical for a hard real-time emergency response.

Emergency/alarm conditions (arrhythmia detection, SpO2 critical-low, flatline) are wired to the topmost nested-interrupt level with interrupts globally re-enabled inside lower-level ISRs (nested-interrupt-enable), so an emergency can interrupt an interrupt.

Continuous waveform/vital-sign logging (the bulk, non-urgent data volume) bypasses the CPU/interrupt path almost entirely: each monitoring device streams into a circular memory buffer via DMA, so storing this data never contends with, delays, or is delayed by emergency-signal processing.

2. Block-Level Data / Control Flow

```
ECG / SpO2 / BP / Temp / Resp sensors (different sample rates) --continuous--> Per-
device buffer --DMA--> Patient Record Ring Buffer (storage), no CPU involvement
Emergency detector (arrhythmia / apnea / critical SpO2) --> Emergency IRQ (Level-0,
highest, nesting-enabled) --> CPU preempts ANY lower ISR immediately
Routine vital-sign sampling (e.g., periodic ECG ISR) --> Level-1..N IRQs, priority
ordered by clinical urgency & sample rate
Nested Interrupt Controller: Level-0 Emergency can preempt Level-1..N mid-execution
(true nesting, not just queueing)
```

Central Nurse-Station link: notified directly from Emergency ISR, independent of DMA logging path

3. Operating Sequence

1. Every monitoring device streams its samples continuously into memory through its own DMA channel, building a full historical patient record with zero CPU intervention for routine storage.
2. Lower-priority device ISRs (e.g., handling a routine SpO₂ or temperature sample) run with interrupts re-enabled for higher levels, so they can be preempted at any instruction.
3. The moment an emergency condition is detected (in hardware/firmware threshold logic or software analysis), the Level-0 emergency interrupt fires and immediately preempts whatever lower-level ISR is running.
4. The emergency ISR performs the minimum necessary — alarm trigger, care-team notification, critical-event flag — then returns control, allowing the preempted lower-priority ISR to resume.
5. Because continuous data storage rides entirely on DMA, it is never the cause of, nor delayed by, emergency-signal servicing — the two concerns are fully decoupled.

4. Interrupt / DMA Priority Assignment

Data / Event Type	Handling Mechanism	Priority Level	CPU Involvement
Emergency / alarm signal	Nested interrupt, preemptive	Level-0 (highest, can nest)	Immediate, minimal ISR
Routine vital-sign sampling	Standard prioritized interrupt	Level-1..N (by urgency/rate)	Short periodic ISR
Continuous waveform logging	DMA to memory ring buffer	N/A (no interrupt on data path)	None (autonomous transfer)

5. Why This Design Meets the Constraints

- Assigning interrupt priority by clinical urgency (not just arrival order) ensures faster/urgent devices are never stuck behind slower ones.
- True nested-interrupt handling lets an emergency preempt an in-progress lower-priority ISR rather than merely being queued behind it, minimizing emergency response latency.
- DMA-based continuous logging removes the largest, steadiest data volume entirely from the interrupt/CPU path, so it can never compete with or delay emergency handling.
- Decoupling storage (DMA) from alarm response (nested IRQ) means high logging rates from many devices cannot degrade the system's real-time emergency guarantees.